



AGENTIC SOFTWARE DEVELOPMENT

**Green means
two different
things.**

#1 in the silent-failure series



THE STAKES



Nothing has failed in weeks. Is that good news?

There is nothing on the
dashboard that separates a
clean run from a check that
stopped looking.



WHAT WE ALL ASSUME



A check has two answers.

Pass or fail. That is how every
runner, every dashboard and
every merge gate is built.

THE BREAK

**It has two answers
and three states.**

nothing is wrong

something is wrong

I could not look

The third one has nowhere to
go, so it comes out as the first.

MEASURED, 1994

46% →
0.4%

Faults that produced a wrong answer instead of stopping, before and after adding behaviour-based error detection.

Madeira et al., FTCS-24 · ~20,000 injected faults, 2 machines · range 2.3–0.4%



THE FIX



The third verdict: I could not look.

A separate exit code meaning the instrument is broken — which is a different finding from the code being broken.

THE METHOD

- 1 Write the case the check exists to catch, as a fixture.
- 2 Run the check against that fixture before every real scan.
- 3 If it misses, exit with an error — not a failure.

THE METHOD

- 4 Never report a result the check could not compute. Silence is not a pass.
- 5 List every check that has only ever returned pass. That list is your blind spot, ranked.

five steps, none of them hard

THE TEST

How would it know if it had stopped working?

Break something a check is supposed to catch, on purpose, and watch it go red. If it stays green, you never had that check.

run it against your own pipeline this week

WHERE THIS STOPS WORKING

A selftest can rot too.

It is one more thing that can quietly stop matching. It moves the blind spot rather than removing it — and a smaller blind spot you can name beats a large one you cannot.

the rule has an edge, and this is it